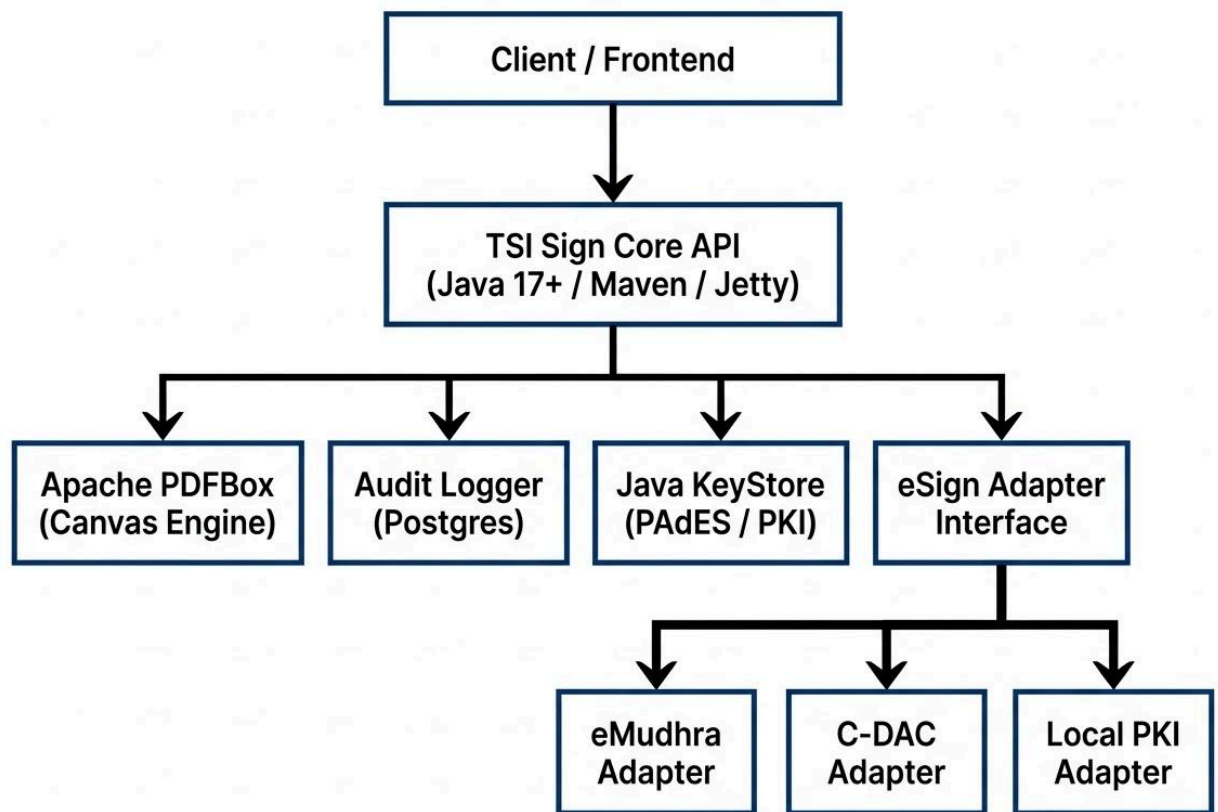


Aligning with **TSI Coop's standard open-source enterprise stack** (JDK 17+, Maven, Eclipse Jetty / Servlet containers, and PostgreSQL), the technical architecture and API schema are restructured below using native Java constructs (Jakarta EE / Spring / Pure Java Driver interfaces).

1. High-Level Technical Architecture



Core Components

- **TSI Core Engine (Java 17 / Apache Maven):** Handles document lifecycle states (DRAFT, PENDING, SIGNED), calculates SHA-256 hashes, and uses libraries like **Apache PDFBox** for visual placement and underlying PDF manipulation.
- **Local PKI Engine (KeyStore / HSM):** Uses native Java Cryptography Architecture (java.security) and KeyStore instance to cryptographically seal PDFs using standard PAdES (.pfx/.p12) signatures locally without calling outside networks.
- **Pluggable Adapter Interface:** A standard Java interface (ESignAdapter) implemented by provider drivers. It handles XML/SOAP or JSON transformations and performs HTTP

handshakes with Certifying Authorities (CAs).

2. Java Adapter Interface & Data Transfer Objects (DTOs)

A. Core Java Interface

Java

Java

```
package org.tsicooop.sign.adapter;

import org.tsicooop.sign.dto.SigningSessionRequest;
import org.tsicooop.sign.dto.SigningSessionResponse;
import org.tsicooop.sign.dto.SigningResult;
import org.tsicooop.sign.exception.ESignAdapterException;

public interface ESignAdapter {

    /**
     * Identifies the unique vendor key (e.g., "emudhra", "cdac",
     "local_pki")
     */
    String getProviderId();

    /**
     * Initiates a remote eSign transaction with the CA Gateway
     */
    SigningSessionResponse initiateSigning(SigningSessionRequest
request) throws ESignAdapterException;

    /**
     * Validates and processes incoming CA webhooks or redirect
responses
     */
    SigningResult processCallback(String rawCallbackPayload) throws
ESignAdapterException;
}
```

B. Data Transfer Objects (Java Record / POJO)

Java

Java

```
package org.tsicoop.sign.dto;

import java.util.Map;

// Session Request DTO
public record SigningSessionRequest(
    String documentId,
    String documentHashSha256,
    SignerIdentity signerIdentity,
    String redirectUrl,
    Map<String, String> providerCredentials // Inject enterprise
    ASP keys/cert paths dynamically
) {
    public record SignerIdentity(
        String name,
        String email,
        String phone
    ) {}
}

// Session Response DTO
public record SigningSessionResponse(
    String transactionId,
    String gatewayUrl,
    Map<String, String> redirectParameters
) {}

// Signing Result DTO
public record SigningResult(
    boolean isSuccess,
    String transactionId,
    String pkcs7Signature, // Encrypted signature payload from CA
    AuditDetails auditDetails,
    String errorCode,
    String errorMessage
) {
    public record AuditDetails(
        String signerName,
        String authType,
        String caIssuer,
        String timestampISO
    ) {}
}
```

3. REST API Schemas (Jackson / JSON Mapping)

A. Initiate Signing Session

- **Endpoint:** POST /api/v1/documents/{documentId}/sign
- **Content-Type:** application/json

JSON

•

JSON

```
{
  "signerId": "usr_987654321",
  "signatureType": "AADHAAR_OTP",
  "providerId": "emudhra",
  "appearance": {
    "pageNumber": 1,
    "positionX": 100.0,
    "positionY": 450.0,
    "width": 200.0,
    "height": 50.0,
    "showTimestamp": true
  }
}
```

- **Response (HTTP 200 OK):**

JSON

JSON

```
{
  "status": "INITIATED",
  "documentId": "doc_123456789",
  "signingSessionId": "sess_abc123xyz",
  "gatewayAction": {
    "actionType": "REDIRECT",
    "url": "https://esign.emudhra.com/eSignGateway/v3/auth",
    "params": {
      "aspId": "ENTERPRISE_ASP_001",
      "txnRef": "sess_abc123xyz",
    }
  }
}
```

```

      "docHash":
      "e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855"
    }
  }
}

```

B. Adapter Callback Handling (Gateway Return)

- **Endpoint:** POST /api/v1/adapters/{providerId}/callback
- **Content-Type:** application/json (or application/x-www-form-urlencoded)

JSON

•

JSON

```

{
  "txnRef": "sess_abc123xyz",
  "status": "SUCCESS",
  "pkcs7Signature": "MIAGCSqGSIB3DQEHAqCAMIACAQEx...",
  "auditData": {
    "signerName": "JOHN DOE",
    "authType": "OTP",
    "caIssuer": "eMudhra CA 2026",
    "timestamp": "2026-08-22T09:27:19Z"
  }
}

```

- **Response (HTTP 200 OK):**

JSON

JSON

```

{
  "status": "COMPLETED",
  "documentId": "doc_123456789",
  "pdfSealed": true,
  "auditTrailId": "aud_777888999"
}

```

C. Local Enterprise KeyStore Sealing (Direct Java PKI)

For internal seals using a .pfx or .jks file located on the local server without hitting third-party APIs.

- **Endpoint:** POST /api/v1/documents/{documentId}/seal-local
- **Content-Type:** application/json

JSON

•

JSON

```
{
  "keyAlias": "tsi_corporate_seal",
  "reason": "Internal Executive Approval",
  "location": "Coimbatore, TN"
}
```

- **Response (HTTP 200 OK):**

JSON

JSON

```
{
  "status": "SIGNED",
  "signatureStandard": "PAdES-B-B",
  "sealedAt": "2026-08-22T09:27:19Z",
  "sha256Checksum": "a8f5f167f44f4964e6c998dee827110c"
}
```

Here is a clean PostgreSQL database schema designed for **TSI Sign**. It uses strong relational integrity, proper JSONB column structures for flexible audit metadata, and BIGINT / UUID keys.

1. Entity-Relationship Schema

Entity-Relationship Structure

- Organizations have many documents.
- Users own many documents.
- Documents have many signers.
- Documents have many seals.
- Documents have many audit logs.
- Signers are associated with seals.

2. DDL Scripts (PostgreSQL)

SQL DDL Script

SQL

```
-- Enable UUID extension
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

-- 1. ORGANIZATIONS
CREATE TABLE organizations (
  org_id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  org_name        VARCHAR(255) NOT NULL,
  slug            VARCHAR(100) NOT NULL UNIQUE,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- 2. USERS
CREATE TABLE users (
  user_id         UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  org_id          UUID NOT NULL REFERENCES organizations(org_id)
ON DELETE CASCADE,
  email           VARCHAR(255) NOT NULL UNIQUE,
  full_name       VARCHAR(255) NOT NULL,
  role            VARCHAR(50) NOT NULL DEFAULT 'USER',
  created_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- 3. DOCUMENTS
CREATE TABLE documents (
  document_id     UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```

    org_id          UUID NOT NULL REFERENCES organizations(org_id)
ON DELETE CASCADE,
    created_by      UUID NOT NULL REFERENCES users(user_id),
    title           VARCHAR(255) NOT NULL,
    file_path       TEXT NOT NULL,
    original_hash   VARCHAR(64) NOT NULL,
    sealed_hash     VARCHAR(64),
    status          VARCHAR(30) NOT NULL DEFAULT 'DRAFT',
    created_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    expires_at      TIMESTAMPTZ
);

```

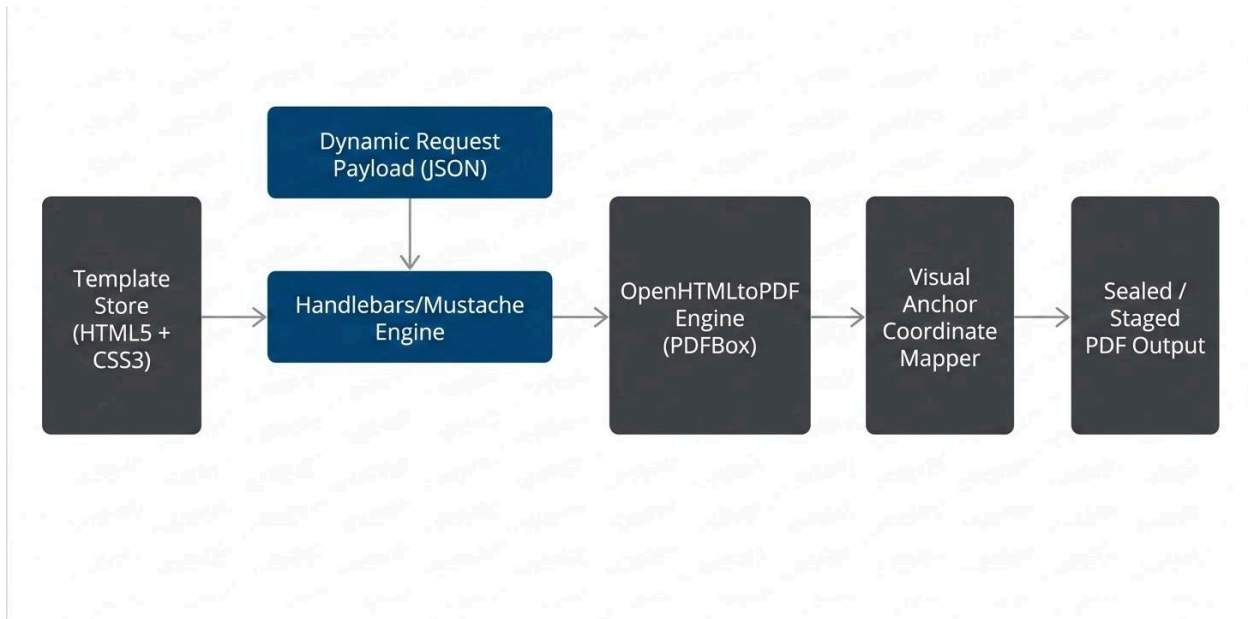
3. Key Architectural Features

- **Data Isolation:** organizations handles multi-tenant boundary separation out-of-the-box.
- **Separation of Concerns:** Signing workflow (document_signers) is strictly decoupled from cryptographic raw output (document_seals).
- **Section 85B Compliance:** The audit_logs table stores the exact state change timeline alongside IP addresses, User-Agent strings, and raw JSON parameters inside metadata to preserve legal presumption in court.
- **Non-Blocking Indexing:** Added indexes on external_txn_ref and document_id for fast callback processing during high-volume gateway redirects.

TSI Sign Document Engine Architecture

The document generation sub-system within **TSI Sign** merges dynamic JSON payloads into HTML/CSS templates, renders raw PDFs, places interactive/visual signature fields, and hands off the execution state to local or external eSign adapters.

1. High-Level Engine Pipeline



2. Core Modules

- **Template Registry (`org.tsicooop.sign.template`):** Stores HTML5/CSS3 document definitions containing variable tags (`{{client.name}}`, `{{loan.amount}}`).
- **Generation Engine (`org.tsicooop.sign.generator`):** Uses **OpenHTMLtoPDF** (backed by **Apache PDFBox 3.x**) to convert HTML+CSS into standard PDF/A documents in pure Java.
- **Field Anchor & Placement Service:** Automatically scans HTML marker tags (e.g., `<div class="tsi-sign-anchor" data-signer="1"></div>`) and translates CSS coordinates into precise PDF page numbers and bounding boxes `$(X, Y, W, H)$`.

3. Java Interface & Class Design

A. Template Engine Service

Java

```
package org.tsicooop.sign.template;
```

```
import java.io.OutputStream;
```

```
import java.util.Map;
```

```
public interface DocumentGeneratorService {
```

```
    /**
```

```
     * Merges dynamic data with an HTML template and compiles a rendered PDF.
```

```
     *
```

```
     * @param templateId Unique handle of the HTML template in DB/Storage
```

```

* @param payloadData JSON key-value pairs to populate inside template
* @param outputStream Target stream where compiled PDF bytes are written
*/
        generatePdf
        ;
    }

```

B. OpenHTMLtoPDF Service Implementation

Java

```

package org.tsicooop.sign.generator;

import com.openhtmltopdf.pdfboxout.PdfRendererBuilder;
import org.tsicooop.sign.template.DocumentGenerationResult;
import org.tsicooop.sign.template.DocumentGeneratorService;

import java.io.OutputStream;
import java.util.Map;

public class OpenHtmlToPdfGeneratorServiceImpl implements DocumentGeneratorService {

    private final TemplateRenderer templateRenderer; // Handlebars / FreeMarker Engine

    public OpenHtmlToPdfGeneratorServiceImpl() {
        this.templateRenderer = templateRenderer;
    }

    @Override
    public generatePdf

    {
        try {
            // 1. Fetch raw HTML string & compile dynamic values
            String rawHtml = fetchTemplateFromStorage(templateId);
            String renderedHtml = templateRenderer.merge(rawHtml, payloadData);

            // 2. Render HTML to PDF via OpenHTMLtoPDF (PDFBox backend)
            PdfRendererBuilder builder = new PdfRendererBuilder();
            builder.useFastMode();
            builder.withHtmlContent(renderedHtml, "");
            builder.toStream(outputStream);
            builder.run();

```

```

// 3. Extract SHA-256 hash and visual signature coordinates
String pdfHash = calculateSha256(outputStream);
var signatureAnchors = extractAnchorPositions(renderedHtml);

returnnew DocumentGenerationResult(true, pdfHash, signatureAnchors);

    } catch (Exception e) {
thrownew RuntimeException("PDF Generation failed for template: " + templateId, e);
    }
}

private      fetchTemplateFromStorage      {
// Fetch HTML template string from DB/Local Directory
return"<!DOCTYPE html><html><body><h1>Agreement for {{client_name}}</h1></body></html>";
}

private      calculateSha256      {
// Cryptographic digest calculation
return"e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855";
}

private java.util.      extractAnchorPositions      {
// Parse custom CSS anchors to map signature boxes
return java.util.List.of();
}
}

```

4. REST API Endpoints

A. Create HTML Template

- **Endpoint:** POST /api/v1/templates
- **Payload:**

JSON

```

{
  "templateName": "Standard Vendor Agreement",
  "category": "LEGAL",
  "htmlContent": "<!DOCTYPE html><html><head><style>body { font-family: sans-serif;
}</style></head><body><h1>Vendor Agreement</h1><p>This agreement is entered into by
<b>{{vendor_name}}</b>.</p><div id='sig_box_1' class='tsi-signature-anchor' data-signer-order='1'
style='width:200px; height:50px;'></div></body></html>"
}

```

```
}
```

B. Generate Document from Template

- **Endpoint:** POST /api/v1/templates/{templateId}/generate
- **Payload:**

JSON

```
{
  "documentTitle": "Agreement_Vendor_ACME_Corp.pdf",
  "payloadData": {
    "vendor_name": "ACME Technologies Pvt Ltd",
    "effective_date": "2026-09-01"
  },
  "signers": [
    {
      "signerOrder": 1,
      "signerName": "Jane Doe",
      "signerEmail": "jane@acme.com",
      "signatureType": "AADHAAR_OTP",
      "anchorElementId": "sig_box_1"
    }
  ]
}
```

- **Response (HTTP 201 Created):**

JSON

```
{
  "documentId": "doc_9988776655",
  "status": "PENDING",
  "originalHashSha256":
    "8f4e3c2b1a0f9e8d7c6b5a4f3e2d1c0b9a8f7e6d5c4b3a2f1e0d9c8b7a6f5e4d",
  "signatureAnchorsMapped": 1,
  "downloadUrl": "/api/v1/documents/doc_9988776655/download"
}
```